Review: Optimization

Rainybunny

2026 年 4 月 1 日

目录

1. Optimization
   - Code Motion
   - Reduction in Strength
   - Share Common Subexpressions
   - Dead-code Elimination *
   - Constant Folding *
   - Induction Variable *
2. Optimization Blockers
   - Procedure Calls
   - Memory Aliasing
   - Bonus: C++ Attribute *
3. Instruction-Level Parallelism
   - Modern CPU Design
   - Unrolling

1. Optimization
   - Code Motion
   - Reduction in Strength
   - Share Common Subexpressions
   - Dead-code Elimination *
   - Constant Folding *
   - Induction Variable *
2. Optimization Blockers
   - Procedure Calls
   - Memory Aliasing
   - Bonus: C++ Attribute *
3. Instruction-Level Parallelism
   - Modern CPU Design
   - Unrolling

Code Motion

- Reduce frequency with which computation performed
  - If it will always produce same result
  - Especially moving code out of loop

```
#define N 127
typedef long i64;
typedef unsigned long u64;

i64 rowSum(i64 ary[N][N], int r) {
  i64 res = 0;
  for (int i = 0; i < N; ++i) {
    res += ary[r][i];
  }
  return res;
}
```

(所有演示代码: https://godbolt.org/z/4bfnTs6Ey)

Code Motion

```
; -Og -masm=att
rowSum:
    movl    $0, %edx
    movl    $0, %ecx
    jmp     .L2
.L3:
    movslq  %esi, %rax
    imulq   $1016, %rax, %rax
    addq    %rdi, %rax              ; ary[r]
    movslq  %edx, %r8
    addq    (%rax,%r8,8), %rcx      ; res+=ary[r][i]
    addl    $1, %edx
.L2:
    cmpl    $126, %edx
    jle     .L3
    movq    %rcx, %rax
    ret
```

Code Motion

```
; -O2 -masm=att
rowSum:
        movslq  %esi, %rax
        xorl    %edx, %edx
        imulq   $1016, %rax, %rax
        addq    %rdi, %rax              ; *p=ary[r]
        leaq    1016(%rax), %rcx
.L2:
        addq    (%rax), %rdx            ; res+=p[i]
        addq    $8, %rax
        cmpq    %rcx, %rax
        jne     .L2
        movq    %rdx, %rax
        ret
```

Reduction in Strength

- Replace costly operation with simpler one
  - Shift, add instead of multiply or divide
- Recognize sequence of products

Reduction in Strength

```
; -Og -masm=att
rowSum:
    movl    $0, %edx                ; i=0
    movl    $0, %ecx
    jmp     .L2
.L3:
    movslq  %esi, %rax
    imulq   $1016, %rax, %rax
    addq    %rdi, %rax              ; ary[r]
    movslq  %edx, %r8
    addq    (%rax,%r8,8), %rcx      ; res+=ary[r][i]
    addl    $1, %edx                ; ++i
.L2:
    cmpl    $126, %edx              ; i<N
    jle     .L3
    movq    %rcx, %rax
    ret
```

Reduction in Strength

```
; -O2 -masm=att
rowSum:
    movslq  %esi, %rax
    xorl    %edx, %edx
    imulq   $1016, %rax, %rax
    addq    %rdi, %rax              ; *p=ary[r]
    leaq    1016(%rax), %rcx        ; *q=p+N
.L2:
    addq    (%rax), %rdx            ; res+=*p
    addq    $8, %rax                ; ++p
    cmpq    %rcx, %rax              ; p!=q
    jne     .L2
    movq    %rdx, %rax
    ret
```

Share Common Subexpressions

- Reuse portions of expressions
- c.f. Code Motion?

```
u64 calc_u(u64 a, u64 b, u64 c, u64 d) {
  return (a + b) * (c + d) + a * c + a * d;
}
```

Share Common Subexpressions

```
; -Og -masm=att
calc_u:
    leaq    (%rdi,%rsi), %rax ; a+b
    leaq    (%rdx,%rcx), %rsi ; c+d
    imulq   %rsi, %rax        ; (a+b)*(c+d)
    imulq   %rdi, %rdx        ; a*c
    addq    %rdx, %rax        ; res+=a*c
    imulq   %rcx, %rdi        ; b*d
    addq    %rdi, %rax        ; res+=b*d
    ret
```

Share Common Subexpressions

```
; -O2 -masm=att
calc_u:
    leaq    (%rsi,%rdi,2), %rax ; (2*a)+b
    addq    %rcx, %rdx          ; c+d
    imulq   %rdx, %rax
    ret
```

Share Common Subexpressions

把 u64 换成 i64 会发生什么?

```
i64 calc_i(i64 a, i64 b, i64 c, i64 d) {
  return (a + b) * (c + d) + a * c + a * d;
}
```

Share Common Subexpressions

把 u64 换成 i64 会发生什么?

```
i64 calc_i(i64 a, i64 b, i64 c, i64 d) {
  return (a + b) * (c + d) + a * c + a * d;
}
```

```
; -O2 -masm=att
calc_i:
    leaq    (%rdx,%rcx), %rax
    addq    %rdi, %rsi
    imulq   %rax, %rsi
    imulq   %rdi, %rdx
    imulq   %rcx, %rdi
    addq    %rdx, %rsi
    leaq    (%rsi,%rdi), %rax
    ret
```

Share Common Subexpressions

`u64` 的运算严格等价于 $\mathbb{Z}/2^{64}\mathbb{Z}$, 编译器能够自由地采用结合律, 分配律等化简表达式.

`i64` 的运算溢出是**未定义行为** (Undefined Behavior, UB). 编译器有权假定用户代码不会引入 UB, 但**不能引入新的 UB 可能**.

E.g. `calc_i(I64_MAX, -I64_MAX + 1, 1, -1)`.

Dead-code Elimination *

- Remove code that does not affect the program results[1]
  - Shrinking program size
  - Reducing resource usage such as the number of bytes to be transferred
  - Allowing the running program to avoid executing irrelevant operations, which reduces its running time

[1] https://en.wikipedia.org/wiki/Dead-code_elimination

Dead-code Elimination *

```
1  void squareNonnegative(i64 a) {
2    i64 b = a * a;
3    if (b < 0) { // gcc: 何意味?
4      printf("%lld␣<␣0?␣WTF?", b);
5    } else {
6      printf("%lld␣>=␣0␣of␣course.", b);
7    }
8  }
```

Dead-code Elimination *

```
; -O2 -masm=att [2]
.LC0:
    .string "%lld >= 0 of course."
squareNonnegative:
    imulq   %rdi, %rdi
    xorl    %eax, %eax
    movq    %rdi, %rsi
    movl    $.LC0, %edi
    jmp     printf
```

[2] -Og 行为一样, -O0 会进行正负判断.

Dead-code Elimination *

- gcc: 相信程序员不会写 UB.

Dead-code Elimination *

- gcc: 相信程序员不会写 UB.
- 程序员: 诶, 我有一计!

Dead-code Elimination *

- gcc: 相信程序员不会写 UB.
- 程序员: 诶, 我有一计!

```
int main() {
  i64 a = (1ll << 61) | (1ll << 1);
  // 刚好溢出到符号位! ohhh!
  squareNonnegative(a);
  return 0;
}
```

Dead-code Elimination *

- gcc: 相信程序员不会写 UB.
- 程序员: 诶, 我有一计!

```
int main() {
  i64 a = (1ll << 61) | (1ll << 1);
  // 刚好溢出到符号位! ohhh!
  squareNonnegative(a);
  return 0;
}
```

- gcc: 地铁老人手机.jpg

Dead-code Elimination *

- gcc: 相信程序员不会写 UB.
- 程序员: 诶, 我有一计!

```
int main() {
  i64 a = (1ll << 61) | (1ll << 1);
  // 刚好溢出到符号位! ohhh!
  squareNonnegative(a);
  return 0;
}
```

- gcc: 地铁老人手机.jpg

```
-9223372036854775804 >= 0 of course.
```

Constant Folding *

- Constant folding is the process of recognizing and evaluating constant expressions at compile time rather than computing them at runtime.

Constant Folding *

```
// -std=c++11
constexpr u64 fibo(u64 n) {
  return n <= 1 ? 1 : fibo(n - 1) + fibo(n - 2);
}

int main() {
    printf("%lld\n", fibo(35));
    return 0;
}
```

Constant Folding *

```
// -std=c++11
constexpr u64 fibo(u64 n) {
  return n <= 1 ? 1 : fibo(n - 1) + fibo(n - 2);
}

int main() {
    printf("%lld\n", fibo(35));
    return 0;
}
```

你说得对...

```
; -O2 -masm=att
    ; ...
    movl    $14930352, %esi
    ; ...
```

Constant Folding *

```
// -std=c++11
constexpr u64 fibo(u64 n) {
  return n <= 1 ? 1 : fibo(n - 1) + fibo(n - 2);
}

int main() {
    printf("%lld\n", fibo(35));
    return 0;
}
```

你说得对...

```
; -O2 -masm=att
    ; ...
    movl    $14930352, %esi
    ; ...
```

但是编译了 16603ms (编译时递归硬算出 `fibo(35)`).

Induction Variable *

- An induction variable is a variable that gets increased or decreased by a fixed amount on every iteration of a loop or is a linear function of another induction variable.

1 Optimization
- Code Motion
- Reduction in Strength
- Share Common Subexpressions
- Dead-code Elimination *
- Constant Folding *
- Induction Variable *

2 Optimization Blockers
- Procedure Calls
- Memory Aliasing
- Bonus: C++ Attribute *

3 Instruction-Level Parallelism
- Modern CPU Design
- Unrolling

Procedure Calls

```
1  __attribute__((noinline))
2  i64 length(const char* str) {
3      i64 res = 0;
4      while (str[res] > 0) ++res; // intended
5      return res;
6  }
7
8  void toLower(char* __restrict__ str) {
9      for (i64 i = 0; i < length(str); ++i) {
10         if ('A' <= str[i] && str[i] <= 'Z') {
11             str[i] += 'a' - 'A';
12         }
13     }
14 }
```

Procedure Calls

```
1  __attribute__((noinline))
2  i64 length(const char* str) {
3      i64 res = 0;
4      while (str[res] > 0) ++res; // intended
5      return res;
6  }
7
8  void toLower(char* __restrict__ str) {
9      for (i64 i = 0; i < length(str); ++i) {
10         if ('A' <= str[i] && str[i] <= 'Z') {
11             str[i] += 'a' - 'A';
12         }
13     }
14 }
```

- Compiler treats procedure call as a black box

Procedure Calls

- inline 一定能拯救吗?

```
1  __attribute__((always_inline)) inline
2  i64 length(const char* str) {
3      i64 res = 0;
4      while (str[res] > 0) ++res;
5      return res;
6  }
```

Procedure Calls

- inline 一定能拯救吗?

```
1  __attribute__((always_inline)) inline
2  i64 length(const char* str) {
3      i64 res = 0;
4      while (str[res] > 0) ++res;
5      return res;
6  }
```

- 不一定.

Procedure Calls

- inline 一定能拯救吗?

```
__attribute__((always_inline)) inline
i64 length(const char* str) {
    i64 res = 0;
    while (str[res] > 0) ++res;
    return res;
}
```

- 不一定.
- length 本身不是关于形参 str 的纯函数, 它还依赖于 str 指向的一段内存. 而 toLower 中的确会修改 str 指向的内存, 编译器未能发现这些修改不会导致 length 的结果改变.

Memory Aliasing

```
1 void i64cpy(i64* dst, const i64* src, u64 n) {
2     for (u64 i = 0; i < n; ++i) {
3         dst[i] = src[i];
4     }
5 }
```

- 程序员: 总不能有人调用 `i64cpy(src, src + 1, n)` 吧?

Memory Aliasing

```
1  void i64cpy(i64* dst, const i64* src, u64 n) {
2      for (u64 i = 0; i < n; ++i) {
3          dst[i] = src[i];
4      }
5  }
```

- 程序员: 总不能有人调用 i64cpy(src, src + 1, n) 吧?
- gcc: 总有神人会调用 i64cpy(src, src + 1, n) 的...

```
1  ; -O2 -masm=att
2  i64cpy(long*, long const*, unsigned long):
3      testq   %rdx, %rdx
4      je      .L25
5      xorl    %eax, %eax
6  .L27:
7      movq    (%rsi,%rax,8), %rcx
8      movq    %rcx, (%rdi,%rax,8)
9      addq    $1, %rax
10     cmpq    %rax, %rdx
11     jne     .L27
12 .L25:
13     ret
```

Memory Aliasing

- 手动消除 aliasing: 用变量暂存计算结果, 减少内存写回;
- 相信现代编译器的指针分析.

Memory Aliasing

- 手动消除 aliasing: 用变量暂存计算结果, 减少内存写回;
- 相信现代编译器的指针分析.
- 直接告诉编译器!

```
1 void i64cpy_restr(i64* __restrict__ dst,
2     const i64* __restrict__ src, u64 n) {
3     for (u64 i = 0; i < n; ++i) {
4         dst[i] = src[i];
5     }
6 }
```

- 程序员: 放心吧, 没有神人.

Memory Aliasing

- 手动消除 aliasing: 用变量暂存计算结果, 减少内存写回;
- 相信现代编译器的指针分析.
- 直接告诉编译器!

```
1 void i64cpy_restr(i64* __restrict__ dst,
2     const i64* __restrict__ src, u64 n) {
3     for (u64 i = 0; i < n; ++i) {
4         dst[i] = src[i];
5     }
6 }
```

- 程序员: 放心吧, 没有神人.
- gcc: 那还说啥? 我看懂了!

```
1 ; -O2 -masm=att
2 i64cpy_restr(long*, long const*, unsigned long):
3     testq   %rdx, %rdx
4     je      .L32
5     salq    $3, %rdx
6     jmp     memcpy
7 .L32:
8     ret
```

Bonus: C++ Attribute *

- 我知道 `exit()` 不会返回, 我知道 `int count` 其实一定是非负整数, 我知道 `isprime(n)` 很可能是 `false`...

[3] https://en.cppreference.com/w/cpp/language/attributes.html

Bonus: C++ Attribute *

- 我知道 `exit()` 不会返回, 我知道 `int count` 其实一定是非负整数, 我知道 `isprime(n)` 很可能是 `false`...
- `gcc`: tell me!
- `[[noreturn]]`, `[[assume(...)]]`, `[[unlikely]]`, ...[3]

[3] https://en.cppreference.com/w/cpp/language/attributes.html

Bonus: C++ Attribute *

- 我知道 `exit()` 不会返回, 我知道 `int count` 其实一定是非负整数, 我知道 `isprime(n)` 很可能是 `false`...
- `gcc`: tell me!
- `[[noreturn]]`, `[[assume(...)]]`, `[[unlikely]]`, ...[3]
- **若 attribute 被违背, 行为未定义.**

[3] https://en.cppreference.com/w/cpp/language/attributes.html

Bonus: C++ Attribute *

```
/* https://zhuanlan.zhihu.com/p/663893518 */
i64 div128(i64 x) {
    return x / 128;
}

// -std=c++23
i64 div128_assume(i64 x) {
    [[assume(x >= 0)]];
    return x / 128;
}
```

```
; -O2 -masm=att -std=c++23
div128(long):
    testq   %rdi, %rdi
    leaq    127(%rdi), %rax
    cmovns  %rdi, %rax
    sarq    $7, %rax
    ret
div128_assume(long):
    movq    %rdi, %rax
    sarq    $7, %rax
    ret
```

1 Optimization
- Code Motion
- Reduction in Strength
- Share Common Subexpressions
- Dead-code Elimination *
- Constant Folding *
- Induction Variable *

2 Optimization Blockers
- Procedure Calls
- Memory Aliasing
- Bonus: C++ Attribute *

3 Instruction-Level Parallelism
- Modern CPU Design
- Unrolling

Mordern CPU Design

- Superscalar Processor

Mordern CPU Design

- Superscalar Processor
- Multiple Functional Units

Mordern CPU Design

- Superscalar Processor
- Multiple Functional Units
- Pipelined Functional Units
  - Latency Bound
  - Throughput Bound

Unrolling: Key Ideas

```
1  void combine4(vec_ptr v, data_t *dest) {
2      long i, length = vec_length(v);
3      data_t *d = get_vec_start(v);
4      data_t t = IDENT;
5      for (i = 0; i < length; i++)
6          t = t OP d[i];
7      *dest = t;
8  }
```

- **连续且无数据依赖的指令**更能充分发挥 CPU 性能, 但是:
  - OP 是结合的, 而 combine4 = foldl (OP) IDENT;
  - OP 是交换的, 而 combine4 只用单变量累加.

Unrolling 2x1

```
void combine_2x1(vec_ptr v, data_t *dest) {
    long length = vec_length(v);
    long limit = length-1;
    data_t *d = get_vec_start(v);
    data_t x = IDENT;
    long i;
    for (i = 0; i < limit; i+=2) {
        x = (x OP d[i]) OP d[i+1];
    }
    for (; i < length; i++) {
        x = x OP d[i];
    }
    *dest = x;
}
```

- 相比原版, `int add` 得到部分优化.

Unrolling 2x1a: Reassociation

```
void combine_2x1a(vec_ptr v, data_t *dest) {
    long length = vec_length(v);
    long limit = length-1;
    data_t *d = get_vec_start(v);
    data_t x = IDENT;
    long i;
    for (i = 0; i < limit; i+=2) {
        x = x OP (d[i] OP d[i+1]);
    }
    for (; i < length; i++) {
        x = x OP d[i];
    }
    *dest = x;
}
```

- 相比 2x1, 其他的 CPE 几乎减半.

Unrolling 2x2: Separate Accumulator

```
void combine_2x2(vec_ptr v, data_t *dest) {
    long length = vec_length(v);
    long limit = length-1;
    data_t *d = get_vec_start(v);
    data_t x0 = IDENT;
    data_t x1 = IDENT;
    long i;
    for (i = 0; i < limit; i+=2) {
        x0 = x0 OP d[i];
        x1 = x1 OP d[i+1];
    }
    for (; i < length; i++) {
       x0 = x0 OP d[i];
    }
    *dest = x0 OP x1;
}
```

- 相比 2x1a, `int add` 得到部分优化.

Unrolling: Result Table

Method	Integer		Double FP	
Operation	Add	Mult	Add	Mult
Combine4	1.27	3.01	3.01	5.01
Unroll 2x1	1.01	3.01	3.01	5.01
Unroll 2x1a	1.01	1.51	1.51	2.51
Unroll 2x2	0.81	1.51	1.51	2.51
Latency Bound	*1.00*	*3.00*	*3.00*	*5.00*
Throughput Bound	*0.50*	*1.00*	*1.00*	*0.50*

Unrolling: Result Table

Method	Integer		Double FP	
Operation	Add	Mult	Add	Mult
Combine4	1.27	3.01	3.01	5.01
Unroll 2x1	1.01	3.01	3.01	5.01
Unroll 2x1a	1.01	1.51	1.51	2.51
Unroll 2x2	0.81	1.51	1.51	2.51
Latency Bound	*1.00*	*3.00*	*3.00*	*5.00*
Throughput Bound	*0.50*	*1.00*	*1.00*	*0.50*

- 进一步优化: 增大展开路数, 激发 functional units 流水化.
- 思考:
  - 为什么刚刚的 unrolling 几乎没有激发流水化?

Unrolling: Result Table

Method	Integer		Double FP	
Operation	Add	Mult	Add	Mult
Combine4	1.27	3.01	3.01	5.01
Unroll 2x1	1.01	3.01	3.01	5.01
Unroll 2x1a	1.01	1.51	1.51	2.51
Unroll 2x2	0.81	1.51	1.51	2.51
Latency Bound	*1.00*	*3.00*	*3.00*	*5.00*
Throughput Bound	*0.50*	*1.00*	*1.00*	*0.50*

- 进一步优化: 增大展开路数, 激发 functional units 流水化.
- 思考:
  - 为什么刚刚的 unrolling 几乎没有激发流水化?
  - 从理论上分析, 几路展开能让 `double mult` 贴近 throughput bound? (2 fp pipelined mult units, latency = 5; 2 fp load units)

Unrolling: Demostration

- CPU: `AMD Ryzen 9 8945HS w/ Radeon 780M Graphics`
- gcc: `gcc version 11.4.0 (Ubuntu 11.4.0-1ubuntu1 22.04.2)`
- target: `x86_64-linux-gnu`
- option: `-Og -masm=att -std=c11`
- test code and script: `https://paste.ubuntu.com/p/kmP4GK7Rg5`

Thanks!